LLM-Primary Gold Annotation for Cyber Threat Intelligence Named Entity Recognition: Independent Evaluation of Token-Classification Architectures*

Daniel Dye
Bellini College of Artificial Intelligence, Cybersecurity, and Computing, University of South Florida
Tampa, Florida, USA
ddye@usf.edu

Abstract

Cyber threat intelligence (CTI) reports contain entity types with no general-domain analogue—ATT&CK technique identifiers, file hashes, TLP markings, threat actor aliases—that off-the-shelf named entity recognition (NER) models handle poorly. Prior work in this space, including an earlier iteration of this project, evaluates against *silver* labels produced by the same class of dictionary/rule annotator being improved upon. We show that this evaluation is circular: a checkpoint that scores 0.74 F1 against dictionary-derived silver labels falls to 0.35 against independently produced gold, because the silver labels share the model's blind spots rather than measuring against them. We instead generate gold labels directly with a frontier LLM (Claude Sonnet), which reads full advisory text and proposes exhaustive entity annotations under per-label span-format validators; a human curates the evaluation split. On this gold we fine-tune three token-classification encoders—RoBERTa-base, RoBERTa-large [7], and SecureBERT [1], a RoBERTa-architecture model domain-pretrained on security text—and evaluate all three with an identical document-level harness, alongside GLiNER [10], a zero-shot span-matching architecture, fine-tuned on the identical gold. SecureBERT performs best (0.71 strict micro-F1), ahead of RoBERTa-base (0.66) and RoBERTa-large (0.65), indicating that domain-adaptive pretraining matters more than parameter count on this task. GLiNER, once retrained on the same independent gold, is the weakest of the four (0.56). This reverses the project's earlier working hypothesis: a prior comparison that scored an *un*retrained GLiNER checkpoint against retrained token classifiers found GLiNER ahead by 9–15 points and attributed the difference to architecture—a comparison that was not controlled for training data and does not hold once it is. All three token classifiers approach ceiling on pattern-structured types (attack technique IDs, CVE identifiers, TLP markings, email addresses) and remain substantially weaker on open-vocabulary semantic types (threat actor group, hacking tool). Extending the comparison to a true zero-shot frontier LLM (Gemini 3.5 Flash) on the same gold, the frontier model surpasses every fine-tuned local model on F1 (0.795 vs. SecureBERT's 0.712), at a cost of 94.5 s mean latency per document and metered per-token API pricing, against SecureBERT's 1.8 s, free, fully offline inference—a 51× speed difference for 8 points of F1. GLiNER, although local, does not share that speed advantage: at 59.7 s per document it is only 1.6× faster than the cloud API and is dominated by SecureBERT on both accuracy and latency. We report these trade-offs directly; the appropriate operating point depends on deployment context rather than on either a "larger models win" or a "local models win" generalization.

Keywords

named entity recognition, cyber threat intelligence, LLM-based annotation, evaluation methodology, domain-adaptive pretraining

1 Introduction

Cyber threat intelligence reports—government advisories, malware analyses, incident write-ups—are the primary vehicle by which defenders share knowledge, yet extracting structured information from them remains largely manual. Named entity recognition over CTI text must handle entity types with no general-domain analogue: ATT&CK technique identifiers, file hashes, TLP markings, YARA rule names, threat actor aliases. Human annotation at scale is expensive in this domain because it requires security expertise, which motivated an earlier iteration of this project to avoid it entirely: build a *silver*-labeled corpus from agreement between a dictionary/regex annotator (STIX indicators, MITRE ATT&CK lookups) and zero-shot GLiNER [10] predictions, and evaluate against held-out silver labels produced by the same dictionary annotator.

That evaluation is circular, and the gap it hides is large. A blind human audit of predicted spans found the dictionary annotator's own gold-recall to be materially incomplete—it misses real entities the dictionary never encoded—so a model that learns to reproduce the dictionary's blind spots is rewarded, not penalized, by silver-label scoring. We confirm this quantitatively: rescoring an existing dictionary-silver-trained checkpoint against independently produced gold drops its score from 0.74 to 0.35 F1 (Section 4). The checkpoints did not degrade; the yardstick changed.

This finding motivates the pipeline described here. Rather than avoiding human involvement entirely, we move it to where it is cheapest to apply correctly: a frontier LLM (Claude Sonnet) reads each advisory's full text and proposes exhaustive gold-standard entity annotations, subject to per-label span-format validators at generation time; a human then curates the evaluation split (accepting, rejecting, or relabeling proposals), while the larger training split remains LLM-only. This approach is not free of human effort, but it replaces exhaustive first-pass annotation with targeted review—and, critically, it produces evaluation gold that is *independent* of any dictionary or rule system a model might learn to imitate.

We ask four research questions:

RQ1 How large is the gap between circular (dictionary-silver) evaluation and independent (LLM-primary, human-curated)

*Code, data, and fine-tuned models are publicly available: https://github.com/dandye/nlp_capstone. See Section 7.

evaluation for a model trained on the old weak-supervision pipeline?

RQ2 Once both training and evaluation labels come from the independent LLM-gold pipeline, does model scale (RoBERTa-base vs. RoBERTa-large) or domain-adaptive pretraining (SecureBERT) matter more for token-classification CTI NER?

RQ3 Which entity types remain hard even after retraining on independent gold, and does the pattern track training-example frequency, structured-vs-semantic label type, or both?

RQ4 How does this fine-tuned token-classification fleet compare to a zero-shot span-matching architecture (GLiNER) retrained on the identical independent gold, and to a frontier LLM run zero-shot?

All four research questions are answered with data trained and scored on the current LLM-gold pipeline: RQ4's architecture half with a GLiNER checkpoint retrained on the identical gold, and its frontier-LLM half with a zero-shot Gemini 3.5 Flash run on the same gold and harness (Section 4.4). The prior iteration's Gemini numbers, scored against dictionary-silver gold, are superseded and not reported.

2 Related Work

GLiNER [10] treats NER as matching natural-language entity-type descriptions against learned span embeddings, enabling zero-shot extraction of arbitrary types; its schema-driven successor GLiNER2 [9] unifies NER with classification and structured extraction in one model. Distantly supervised NER was formalized for open-domain text by BOND [6], which fine-tunes a BERT-based token classifier [4] on knowledge-base-derived labels and then denoises them through a teacher–student self-training loop. The failure modes BOND targets—incomplete and noisy weak labels—are the same failure modes that motivated this project's move away from dictionary supervision.

Cybersecurity-specific NER corpora suffer acutely from schema fragmentation: Schmidt et al. [8] show that naively unifying four CTI datasets under a coarse shared label set *hurts* performance relative to single-dataset training, while Ech-Chammakhy et al. [5] demonstrate that principled, STIX-anchored harmonization recovers a 30% relative F1 gain over the same naive union—evidence that *how* labels are unified matters as much as how much data is unified. Our LLM-primary annotation pipeline sidesteps unification entirely by defining the label schema once and generating gold directly for it, rather than merging pre-existing dictionary schemas. Deka et al. [3] extend the entity-type inventory to attribution-specific concepts (impact, course-of-action) with deliberately long, sentence-level spans, and find instruction-fine-tuned LLMs outperform encoder taggers on their benchmark, motivating our use of an LLM as the *annotator* rather than the deployed extractor—although Section 4.4 finds the same LLM-outperforms-encoder pattern holds even for a deployed zero-shot extractor on our corpus, so that choice is ultimately one of deployment cost rather than of where an LLM can add value. Alam et al. [2] benchmark LLMs directly on CTI tasks including entity extraction, providing the frontier-model baseline methodology that RQ4 adapts.

LLM-as-annotator is itself an active line of work distinct from LLM-as-extractor: rather than running a frontier model zero-shot at inference time, we use it once, offline, to produce training and evaluation labels for smaller, cheaper, deployable encoders—a distillation-flavored setup, except that the teacher's output is explicitly checked (span-format validators, human curation) rather than trusted directly. The failure mode being guarded against is exactly the one BOND and the dictionary-silver approach share: an annotator's systematic blind spots being learned uncritically by the student.

3 Data and Method

3.1 Pipeline

We scrape approximately 5,000 CISA cybersecurity advisories; each is converted from PDF to markdown via docling, then stripped of layout artifacts and normalized (page-furniture deduplication, pagination removal, split-hash rejoining) to a clean, model-facing text version. For 82 of these advisories we generate gold entity annotations across 16 CTI-specific labels[1] directly with a frontier LLM (Claude Sonnet): the model reads each advisory's full clean text and proposes exhaustive entity spans, which are verbatim-matched back to the source text and rejected if they do not match exactly, and further gated by per-label span-format validators (hash length and character set, CVE/CWE/ATT&CK technique-ID grammar, IP/email format including defanged forms, TLP enumeration) so that structurally malformed proposals never enter the gold set. This is an annotation-time generation gate, not a post-hoc filter.

The 66 training-split advisories receive LLM annotation only. The 16 validation/test advisories additionally pass through human curation via a purpose-built review tool: a human accepts, rejects, or relabels each LLM proposal and may add entities the LLM missed, including an `UNK` category (not used for training or evaluation directly) for real entities that fit no current label, which we use to identify candidate label types. Evaluation gold is therefore human-verified; training gold is not. We report this as a limitation (Section 5) but consider it acceptable because the LLM-primary approach's failure mode—proposing an entity a human would reject—is structurally different from the dictionary approach's failure mode of *never proposing* entire classes of entity for which it has no rule, and a human-verified evaluation set is positioned to catch either.

The training split contains 6,701 gold entities across 66 advisories; the evaluation split (8 validation, 8 test advisories) contains 1,995 gold spans in the test set alone once chunked for scoring (Section 3.4). This is roughly 3.5× the raw entity volume of the prior dictionary-silver pipeline's test set on the same page count, because dictionary/regex annotation structurally omits several semantic label types (e.g., *affected software product*, *hacking tool*) that an LLM reader identifies from context.

3.2 Split

We split by advisory *document*, not by chunk, before chunking for training—66 advisories train / 8 validation / 8 test—so that overlapping text windows from the same document cannot appear on both sides of the split. This is a deliberate fix carried over from

[1] affected software product, attack technique id, attack technique name, cve identifier, cwe identifier, email address, file hash, file name, file path, hacking tool, ip address, malware family, tactic, threat actor group, tlp classification, yara rule name.

the prior pipeline iteration: chunk-level splitting leaks document context (shared boilerplate, repeated entity mentions) across the train/test boundary and inflates apparent performance.

3.3 Models

RoBERTa-base and RoBERTa-large [7]. Standard BIO token classifiers fine-tuned on the LLM-gold training split. Because gold spans are word-level and RoBERTa's tokenizer is subword-based, we tag the first wordpiece of each word and mask continuation wordpieces from the loss. Corpus chunks vary widely in length, and long or hash-heavy chunks fragment into more wordpieces than a single window admits under naive truncation; we instead tokenize with overlapping sliding windows (a 510-wordpiece budget per window, 50-word overlap between windows) so that every word is covered by at least one window, and take each word's predicted tag from the window in which it sits most centrally at inference time. RoBERTa-base trains for 8 epochs at learning rate 2e-5, effective batch size 16; RoBERTa-large uses the identical recipe with effective batch size 32 (gradient accumulation, to fit the larger model). Length-grouped batching reduces padding waste across the wide chunk-length range.

SecureBERT [1]. The same RoBERTa architecture and fine-tuning recipe as RoBERTa-base (8 epochs, learning rate 2e-5, effective batch size 16), but initialized from a checkpoint further pretrained on a security-text corpus rather than general-domain text. Because architecture and fine-tuning recipe are held identical to RoBERTa-base, any difference between the two isolates the effect of domain-adaptive pretraining from the effect of the fine-tuning data or procedure.

GLiNER [10]. `urchade/gliner_medium-v2.1`, zero-shot span-matching, fine-tuned on the identical LLM-gold training split: 5 epochs, learning rate 5e-5, effective batch size 32 (batch 8, gradient accumulation 4), best checkpoint selected by validation loss (GLiNER's trainer reports no span-level F1 during training; that is computed separately with the same harness used for the token classifiers). Unlike the token classifiers, GLiNER handles its own tokenization and length limits internally rather than using the sliding-window scheme of Section 3.4; several of the longest test-set chunks were truncated at its 384-token limit, which we record as a limitation (Section 5) because it plausibly disadvantages GLiNER specifically on labels that skew toward the end of long chunks.

Frontier LLM.. Gemini 3.5 Flash, run zero-shot with no fine-tuning, following Alam et al. [2]. Full documents are covered by overlapping 15,000-character windows (500-character overlap) so that no input is truncated, in contrast to GLiNER's fixed token limit above; per-window entity proposals are merged by exact text-and-label match before scoring, since a single entity mention can legitimately recur across overlap regions. Reasoning is disabled (`thinking_budget=0`): NER requires no reasoning tokens, and hidden reasoning output made early informal runs both slow and expensive.

3.4 Evaluation

All models are scored with the same document-level harness (`ner.evaluate`): predictions from overlapping chunks are merged back to document-global token indices before matching, so that entities in chunk-overlap regions are not double-counted. We report strict (exact span-and-label match) and relaxed (label match plus span overlap) micro/macro precision, recall, and F1 on the 8-document test split, using the identical scoring implementation across models so that the comparison in Section 4 is not an artifact of different matching rules.

Model	Circular F1	Indep. P	Indep. R	Indep. F1
RoBERTa-base	0.740	0.86	0.22	0.351
SecureBERT	0.755	0.87	0.25	0.393
GLiNER (v1, ref.)	0.666	0.65	0.46	0.540

Table 1: Checkpoints trained on the prior dictionary-silver pipeline, scored against the silver labels they were trained toward ("Circular") vs. against independent LLM+human gold ("Indep."), strict span-and-label match, 8-document test split.

4 Results

4.1 RQ1: Circular vs. Independent Evaluation

Table 1 rescores the prior pipeline's checkpoints—trained on dictionary-silver labels, previously reported at 0.74 and 0.76 F1—against the current, independently produced LLM+human gold, on the same 8 test advisories (1,995 gold spans) used throughout this paper. Both token-classification checkpoints collapse: RoBERTa-base from 0.740 to 0.351, SecureBERT from 0.755 to 0.393, roughly a 47–48% relative drop. The models did not get worse; the measurement stopped being circular. These checkpoints had never been evaluated against labels their own dictionary annotator did not also produce, so silver scoring rewarded them for learning the dictionary's blind spots along with its correct patterns. For reference, the old GLiNER checkpoint (fine-tuned on the same dictionary-silver labels, not yet retrained on the current gold) drops by a smaller margin, 0.666 to 0.540 (19% relative)—the asymmetry that motivated an earlier, informal version of this project's RQ4 hypothesis: that span-matching architectures generalize better than token classifiers once evaluation is no longer circular. Section 4.4 retrains GLiNER on the identical gold and shows that the smaller drop was not architectural: it reflects GLiNER's old training data happening to be less circular than the token classifiers' relative to *this specific independent evaluation set*, not an architecture effect that survives a controlled retrain.

4.2 RQ2: Scale vs. Domain Pretraining Under Independent Gold

Table 2 reports the three encoders after retraining on the LLM-gold training split and scoring against the same independent test gold. SecureBERT performs best (0.712 strict / 0.758 relaxed micro-F1), ahead of RoBERTa-base (0.659 / 0.683) and, notably, ahead of RoBERTa-large (0.648 / 0.697) despite RoBERTa-large having roughly 3× the parameters. Domain-adaptive pretraining recovers more of the independent-gold quality here than raw scale does. RoBERTa-large's higher relaxed F1 and macro-F1 alongside a *lower* strict micro-F1 than RoBERTa-base is explained by the per-label

Model	P	R	F1	F1 (relaxed)
RoBERTa-base	0.71	0.61	0.659	0.683
RoBERTa-large	0.71	0.60	0.648	0.697
SecureBERT	**0.73**	**0.70**	**0.712**	**0.758**
GLiNER	0.51	0.61	0.56	0.58

Table 2: Architecture comparison, identical LLM-gold training and evaluation labels, identical evaluation harness. Micro P/R/F1, 8-document test split, 1,995 gold spans.

pattern in Section 4.3: RoBERTa-large predicts the correct general region and type more consistently across labels (helping macro and relaxed matching) but regresses sharply on the single highest-volume label, *malware family*, which drags support-weighted strict micro-F1 down.

Comparing back to Table 1, retraining on independent gold closes most of the gap the circular evaluation had hidden. RoBERTa-base's score against independent gold rises from 0.351 (old checkpoint) to 0.659 (retrained)—most of the way back to its old, circular 0.740—and SecureBERT's independent-gold score (0.712) nearly matches its old circular number (0.755). Correcting *both* the training labels and the evaluation labels recovers almost all of the quality the circular number implied was already present; correcting only the evaluation labels (Table 1) is what made the models appear to regress.

Once retrained on this same gold, GLiNER (0.56 strict / 0.58 relaxed) trails every token classifier, including RoBERTa-large. Its macro-F1 (0.62 strict) sits well above its own micro-F1, the same pattern RoBERTa-large shows above—reasonably consistent type coverage dragged down by weak performance on a few high-volume labels, most strikingly *ip address* (F1 0.03, recall 0.02; Section 4.4 discusses possible causes). This directly answers the question raised in the RQ1 discussion: the old GLiNER checkpoint's smaller circular-vs-independent drop (Table 1) was not evidence of an architectural generalization advantage. Retrained under the identical procedure applied to the token classifiers, GLiNER is the weakest of the four models on this task.

4.3 RQ3: Which Entity Types Remain Hard

Table 3 breaks the three retrained encoders down by label. Structured, pattern-shaped labels are near-solved regardless of frequency: *attack technique id*, *cve identifier*, *tlp classification*, and *email address* all score 0.92–1.00 F1 across all three models, and even the lowest-support label in the test set (*cwe identifier*, $n = 4$) reaches 1.00 on every model—although at that support level a single miss would swing the number by 0.25, so this particular result should be read as indicative rather than as evidence that CWE identification is solved.

The weakest labels are not simply the rarest ones. *Malware family* has the largest test-set support of any label ($n = 438$) yet tops out at 0.65 F1 (SecureBERT); *threat actor group* ($n = 157$) and *hacking tool* ($n = 139$) similarly lag well behind the structured labels despite moderate-to-high support. Frequency alone therefore does not explain the gap: these are open-vocabulary, context-dependent labels (whether a string names a piece of malware, a named intrusion set, or a legitimate administrative tool such as PowerShell often depends on surrounding sentence context rather than surface form), and that semantic ambiguity appears to be a harder problem for a token classifier than sparse supervision is. RoBERTa-large's regression on *malware family* specifically (0.35 vs. 0.59 for RoBERTa-base and 0.65 for SecureBERT, on identical training data) is the single largest per-label swing in the table and the main driver of RoBERTa-large's lower strict micro-F1 in Table 2.

Label	*n*	Base	Large	SecBERT
malware family	438	0.59	0.35	0.65
attack technique id	297	0.86	0.91	**0.99**
affected software product	159	0.48	0.57	0.61
threat actor group	157	0.29	0.42	0.34
attack technique name	144	0.57	0.53	0.57
hacking tool	139	0.37	0.44	0.51
ip address	130	0.85	0.85	0.92
tactic	111	0.59	0.63	0.62
file path	108	0.84	0.83	0.84
file hash	93	0.70	0.65	0.64
file name	89	0.76	0.81	0.74
cve identifier	55	0.92	0.96	0.94
tlp classification	36	0.97	0.97	0.97
email address	26	0.98	0.98	0.98
yara rule name	9	0.62	0.71	0.50
cwe identifier	4	1.00	1.00	1.00
Micro	1995	0.659	0.648	**0.712**

Table 3: Per-label strict F1, sorted by test-set support. Base = RoBERTa-base, Large = RoBERTa-large, SecBERT = SecureBERT.

4.4 RQ4: Architecture Comparison

GLiNER was fine-tuned on the identical LLM-gold training split used for the three token classifiers (5 epochs, learning rate 5e-5, effective batch size 32; best checkpoint by validation loss) and scored with the identical harness on the identical 8-document test split. Table 2's result is unambiguous: GLiNER (0.56 strict F1) is the weakest of the four architectures tested, behind RoBERTa-large (0.65), RoBERTa-base (0.66), and SecureBERT (0.71). This resolves the question RQ1 raised, and the project's prior working hypothesis does not survive controlled retraining. Section 4 sketches why: the old GLiNER checkpoint's smaller circular-vs-independent drop reflected its old training data's relationship to *this* evaluation set, not a property of span-matching architecture that a fair retrain preserves.

Table 4 shows GLiNER is not uniformly weak—it matches or nearly matches the token classifiers on several structured labels (attack technique id 0.99, cwe identifier 1.00, cve identifier 0.96)—but is markedly worse on *file path* (0.42 vs. 0.83–0.84 for the token classifiers) and severely degraded on *ip address* (0.03 vs. 0.85–0.92), a label none of the three token classifiers found difficult. We do not have a confident explanation for the *ip address* collapse. One contributing factor we can document, though not fully attribute, is that GLiNER handles its own tokenization and truncates at a 384-token limit rather than using the token classifiers' sliding-window scheme (Section 3): several of the longest test-set chunks were

Label	GLiNER F1	Best token clf. F1
ip address	0.03	0.92 (SecBERT)
file path	0.42	0.84 (Base/SecBERT)
threat actor group	0.29	0.42 (Large)
hacking tool	0.38	0.51 (SecBERT)
malware family	0.47	0.65 (SecBERT)
file name	0.46	0.81 (Large)
tactic	0.56	0.63 (Large)
attack technique name	0.50	0.57 (Base/SecBERT)
attack technique id	0.99	0.99 (SecBERT)
cve identifier	0.96	0.96 (Large)
cwe identifier	1.00	1.00 (all)

Table 4: GLiNER strict F1 vs. the best-performing token classifier per label (selected labels; full 16-label GLiNER results in `data/gold_v2/results/trained-gliner-v2.json` in the project repository, Section 7).

Model	Strict F1	Relaxed F1	Avg. latency/doc
Gemini 3.5 Flash	**0.795**	**0.803**	94.5 s (7.2–538.7 s)
SecureBERT	0.712	0.758	**1.8 s** (0.4–5.2 s)
RoBERTa-large	0.648	0.697	7.8 s (1.1–30.1 s)
RoBERTa-base	0.659	0.683	—
GLiNER	0.560	0.580	59.7 s (12.3–171.6 s)

Table 5: All five architectures, identical LLM-gold evaluation, identical harness, 8-document test split. Latency is wall-clock document-level inference on identical hardware (Apple Silicon, no batching) for local models, and real API round-trip time for Gemini (Section 3.4)—not directly comparable across the network/local boundary in a strict sense, but both are genuine end-to-end costs a deployment would pay. RoBERTa-base latency was not re-measured because its local checkpoint was not preserved (see text).

silently truncated during evaluation, which could plausibly cost recall on labels that cluster late in long chunks (indicator tables often appear near the end of CISA advisories). We have not verified that IP addresses specifically follow that positional pattern here, and we flag this as a concrete follow-up rather than a settled explanation.

4.5 RQ4 (continued): Frontier-LLM Comparison

Table 5 compares Gemini 3.5 Flash, run zero-shot with no fine-tuning, against the best fine-tuned local model (SecureBERT) on the identical test set and harness. Gemini wins on quality: 0.795 strict / 0.803 relaxed micro-F1, roughly 8 points above SecureBERT's 0.712 / 0.758, and macro-F1 shows the same ordering (0.791 vs. 0.738)—the advantage is not an artifact of a few high-volume labels dominating a support-weighted average. This outcome is the inverse of the project's original premise (that a small fine-tuned model could match a frontier model on a narrow domain task), and we report it as measured.

The cost is real and asymmetric, and—unexpectedly—it is not evenly distributed across the local models. Table 5 measures wall-clock inference on the same 8 documents, on the same hardware (Apple Silicon, CPU/MPS, no batching), for every model in this paper. SecureBERT is not only the most accurate local model; it is also by far the fastest, averaging 1.8 s per document—51× faster than Gemini, with no network call, no per-token billing, no rate limits, and no marginal cost once the checkpoint is downloaded. RoBERTa-large is a step down on both axes (0.648 F1, 7.8 s/document, 12× faster than Gemini). GLiNER, however, departs from the expected pattern: despite running entirely locally, it averages 59.7 s per document—only 1.6× faster than a cloud API call, and dramatically slower than either token classifier. Span-matching against a label-embedding space is architecturally more expensive per token than a per-token softmax head, and that cost is visible directly here: GLiNER is dominated on *both* accuracy and speed by SecureBERT, without the "less accurate but free and fast" trade-off that would make it a defensible choice on locality grounds. Whether Gemini's F1 advantage over SecureBERT justifies its cost depends entirely on deployment context—a security operations center triaging thousands of advisories per day cannot absorb cloud-API latency and cost at that multiplier, whereas a one-off deep analysis of a handful of critical advisories plausibly can—but for GLiNER specifically there is no such trade-off to weigh: SecureBERT surpasses it on every axis measured. RoBERTa-base's original local checkpoint was not preserved for re-measurement (only its evaluation results were retained); given that its architecture and training recipe are identical to RoBERTa-large's, its latency is expected to fall in the same range but is not independently confirmed here.

Two pipeline defects surfaced and were fixed while producing this comparison; both are worth documenting because they illustrate the same lesson as RQ1: an unverified evaluation harness produces confidently wrong numbers. First, span extraction matched entity text against a 4000-character truncation window left over from an earlier, non-chunked inference mode; with full-document chunked inference, this silently dropped roughly 72% of Gemini's correctly identified entities before scoring ever saw them (732 raw entities → 203 matched spans). Corrected to match against the full document, spans recovered from 203 to 2,318 (gold, like this pipeline, credits every occurrence of a repeated entity mention, not only the first). Second, frontier scoring used a parallel, independently written implementation of strict P/R/F1 instead of the shared `ner.evaluate.Scorer` used for every other model in this paper—mathematically equivalent once checked, but a maintenance hazard and a literal violation of this paper's own "identical scoring implementation across models" claim (Section 3.4). Both defects are fixed in the released pipeline; the numbers above reflect the corrected code.

5 Limitations

Training gold is LLM-only, not human-verified. Human curation is applied to the 16 validation/test advisories but not to the 66 training advisories (Section 3), for cost reasons. If Claude Sonnet's proposals have systematic blind spots of their own—the same structural risk that motivated moving off the dictionary annotator, with a different annotator—training labels would inherit them even though the evaluation labels, being human-curated, would not. The results in Section 4 should therefore be read as "evaluated without circularity,"

not as "free of all annotator bias"; only the measurement side of that distinction is currently guaranteed.

Blind gold audit not yet complete. A blind, model-identity-hidden judgment sheet of 300 predicted spans (`paper/capstone/gold_audit.md`) is prepared but not yet human-scored. Once complete, it will report per-model precision independent of both the dictionary-silver noise that motivated this pipeline change and any residual bias in the LLM-primary gold itself, since a human judges each span directly against the source text rather than against either annotator's labels.

Test set size. Eight held-out advisories (1,995 gold spans in the test set) is larger than the prior pipeline's test set (1,442 dictionary-silver spans) and covers label types dictionary annotation structurally could not, but the lowest-support labels (*cwe identifier*, $n = 4$; *yara rule name*, $n = 9$) still have too few examples for their per-label numbers to be read as precise.

Windowing artifacts. All three token classifiers resolve each word's tag from whichever sliding window contains it most centrally at inference time, which is a heuristic rather than a principled ensembling method; a small fraction of predictions near window boundaries may be affected.

GLiNER truncation, not windowing. Unlike the token classifiers, GLiNER's own tokenizer truncates rather than sliding-windows long input, and several of the longest test-set chunks were truncated at its 384-token limit during evaluation. We do not currently know how much of GLiNER's per-label weakness (Section 4.4), particularly the sharp *ip address* drop, is attributable to this truncation versus a genuine architecture or data effect; disentangling the two—e.g., by re-chunking GLiNER's input at a shorter budget matching its limit, mirroring the token classifiers' windowing—is left to future work.

Single frontier model, single sample, one configuration. Section 4.4 compares against one frontier model (Gemini 3.5 Flash) at one point in time with one prompt; we did not sweep prompt variants, temperature, or few-shot examples, and a different frontier model or prompt could plausibly close some of the 8-point F1 gap. Cost and latency figures are similarly a single measurement (one API tier, one region, one day's network conditions), not a stable production benchmark across load or over time.

6 Conclusion

Evaluating against labels produced by the same annotator being improved upon is circular, and the gap it hides is not small: a checkpoint that scored 0.74–0.76 F1 against dictionary-silver gold scores 0.35–0.39 against independently produced gold (RQ1). Retraining on that same independent gold recovers most of the apparent quality—SecureBERT reaches 0.71 F1, nearly matching its own old circular number—showing that the earlier result was not fabricated so much as measured against the wrong yardstick (RQ2). Domain-adaptive pretraining (SecureBERT) recovers more independent-gold quality than raw parameter count (RoBERTa-large), which notably regresses on the single highest-volume label rather than improving on it. Structured, pattern-shaped entity types are essentially solved by all three encoder architectures; open-vocabulary, context-dependent types—most visibly *malware family* and *threat actor group*—remain the hard core of this task regardless of training volume (RQ3).

The architecture question this project set out to answer—whether GLiNER's span-matching design generalizes better than a token classifier's softmax head once measured without circularity—is answered, and not in the direction the project's earlier, uncontrolled comparison suggested. Retrained on the identical independent gold and scored with the identical harness, GLiNER (0.56 F1) is the weakest of the four architectures tested, behind every token classifier including RoBERTa-large (RQ4). The apparent GLiNER advantage in Table 1 was an artifact of comparing a retrained token classifier against an *un*retrained span-matching model—a reminder that the same circularity discipline this paper applies to silver-vs-independent gold (RQ1) must be applied equally strictly to any two checkpoints being compared, or a training-data confound can masquerade as an architecture finding.

Extending the comparison to a frontier LLM completes the picture: zero-shot Gemini 3.5 Flash exceeds every fine-tuned local model on F1, including SecureBERT (0.795 vs. 0.712 strict), at the cost of 94.5 s mean latency per document and metered per-token API pricing against SecureBERT's 1.8 s, free, offline inference—51× faster for 8 fewer points of F1 (RQ4). That trade-off is at least legible: a straightforward speed-and-cost-for-quality exchange whose resolution depends on deployment context. GLiNER's position admits no such justification: at 59.7 s per document it forfeits nearly all of the local-deployment speed advantage (1.6× over Gemini, not 51×) while also trailing SecureBERT on F1, so on this task, measured this way, it is dominated outright rather than trading accuracy for speed. Producing this comparison rigorously also required catching and fixing two evaluation-harness defects (Section 4.4) that would otherwise have quietly misreported it—the same discipline RQ1 required of the silver-vs-independent gold comparison, applied to a frontier-vs-local one. The blind gold-audit results (Section 5) remain outstanding and will be reported once available.

7 Artifact and Data Availability

All code, gold data, and evaluation artifacts required to reproduce the results in this paper are publicly available.

Code and data. The complete pipeline—CISA scraper, text normalization, LLM annotation (`ner.llm_gold`), human-curation webapp (`ner.hitl_ner`), training (`ner.finetune_roberta`, `ner.finetune_gliner`), frontier comparison (`ner.frontier_infer` / `ner.frontier_extract` / `ner.frontier_score`), and the shared evaluation harness (`ner.evaluate`)—is available at https://github.com/dandye/nlp_capstone. Gold annotations are under `data/gold_v2/` (evaluation, human-curated) and `data/gold_v2_train/` (training, LLM-only); the exact train/validation/test partition is `data/gold_v2/split_full.json`. The per-model scores reported in every table are committed as JSON under `data/gold_v2/results/` (`trained-*.json` for models retrained on the current gold, `rescored-*.json` for the RQ1 circularity rescoring) and `data/frontier_results/` (Gemini entities, spans, and scores).

Models. The fine-tuned checkpoints are published on the Hugging Face Hub: https://huggingface.co/pzryathzsdhc/cti-ner-roberta-

large, https://huggingface.co/pzryathzsdhc/cti-ner-securebert, and https://huggingface.co/pzryathzsdhc/cti-ner-gliner. A demonstration notebook, `notebooks/hf_models_demo.ipynb`, downloads the published checkpoints and reproduces windowed inference on sample advisory text; the Colab notebooks under `notebooks/` reproduce the training runs themselves.

References

[1] Ehsan Aghaei, Xi Niu, Waseem Shadid, and Ehab Al-Shaer. 2022. SecureBERT: A Domain-Specific Language Model for Cybersecurity. In *International Conference on Security and Privacy in Communication Systems*.

[2] Md Tanvirul Alam, Dipkamal Nguyen, Amit Bhusal, and Nidhi Rastogi. 2024. CTIBench: A Benchmark for Evaluating LLMs in Cyber Threat Intelligence. In *NeurIPS 2024 Datasets and Benchmarks Track*. arXiv:2406.07599

[3] Pritam Deka, Sampath Rajapaksha, Ruby Rani, Amirah Almutairi, and Erisa Karafili. 2024. AttackER: Enhancing Cyber-Attack Attribution with a Named Entity Recognition Dataset. arXiv:2408.05149

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805

[5] Yasir Ech-Chammakhy, Anas Motii, Youness Rabii, Adnane Azrara, and Yassine Chbili. 2025. CyberNER: A Harmonized STIX Corpus for Cybersecurity Named Entity Recognition. arXiv:2510.26499

[6] Chen Liang, Yue Yu, Haoming Jiang, Siawpeng Er, Ruijia Wang, Tuo Zhao, and Chao Zhang. 2020. BOND: BERT-Assisted Open-Domain Named Entity Recognition with Distant Supervision. In *Proceedings of KDD 2020*. arXiv:2006.15509

[7] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).

[8] Erik Schmidt, Dominika Jalocha, and Mark Michelseen. 2025. Label Unification for Cross-Dataset Generalization in Cybersecurity Named Entity Recognition. arXiv:2507.13870

[9] Urchade Zaratiana, Gil Pasternak, Oliver Boyd, Edward Hurn-Maloney, and Ash Lewis. 2025. GLiNER2: An Efficient Multi-Task Information Extraction System with Schema-Driven Interface. arXiv:2507.18546

[10] Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. 2024. GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer. In *Proceedings of NAACL 2024*. arXiv:2311.08526